Seeing the Symphony

Data Model Reference Document

Raffaele Cipriano · Washburn University · Version 0.1 — March 2026

1. Overview

This document defines the complete data model for Seeing the Symphony. It specifies every object class, its properties, its functions, and its relationships to other objects. It also defines the three persistent file types used by the system and the complete parsing pipeline from MusicXML input to render-ready output.

See: diagram_datamodel_uml.svg for the UML class diagram.

See: diagram_pipeline.svg for the parsing pipeline and file structure diagram.

2. Design Principles

- Concert pitch only — NoteObject always stores concert pitch. Transposition is applied during parsing and never appears in the data model.
- No color in the data model — color is a UI/rendering decision. The data model stores musical facts (circle_of_fifths_pos, instrument_family, technique) that the UI maps to colors.
- Flat collections for performance — notes, sections, spans, and tempo events are stored as flat sorted lists, not trees. The renderer iterates in O(n) without traversal overhead.
- Hybrid time coordinates — every time-positioned object carries both metric coordinates (bar, beat, subdivision 0-60) for human reference and temporal coordinates (time_onset, time_duration in seconds) for the renderer.
- Parse once, render many — all expensive computations (span membership, time_onset conversion, variant index assignment) happen at parse time. The renderer reads pre-computed values.
- Cascading configuration — rendering parameters follow a three-level cascade: System Defaults → Conductor Profile → CIL file. Higher levels override lower ones.
- Separation of concerns — ScoreData contains only musical data. Visualization preferences live in the Conductor Profile and CIL file.

3. File Structure

The system uses three persistent file types. All are external to the ScoreData object, which exists only in memory.

3.1 MusicXML File (.xml / .mxl)

The source score. Read-only. Never modified by the system. Contains pitch, rhythm, dynamics, articulation, tempo markings, and instrument definitions as encoded by the score editor (Sibelius, Dorico, MuseScore, etc.). Quality varies by edition and editor.

3.2 Conductor Profile (.json)

Personal visualization preferences. Score-independent — one file per conductor, reused across all scores. Acts as Level 2 in the configuration cascade. Pre-populated with system defaults on first run.

Conductor Profile — Contents

- template: enum PIANO_ROLL | SPIRAL
- vertical_mode: enum PITCH_BASED | SCORE_BASED
- collision_strategy: enum EXPAND | ALTERNATE | BLEND
- contour_pct_default: float 0.0–1.0
- variant_seed: int
- family_pair_rules: dict[string → float]
- dynamic_tables: dict[instrument_family → dict[DynamicMarkingEnum → float]]

3.3 CIL File — Conductor Interpretation Layer (.json)

Score-specific conductor decisions. Acts as Level 3 in the configuration cascade — overrides the Conductor Profile where present. One file per score per conductor. Auto-saved on every meaningful UI change. Embeds the musicxml_hash of the associated MusicXML file for sync detection.

CIL File — Contents

- musicxml_hash: string — used to detect MusicXML file changes
- note_annotations: dict[note_id → annotation overrides]
- dynamic_span_overrides: dict[span_id → conductor adjustments]
- section_labels: dict[section_id → label + confirmation status]
- tempo_overrides: dict[tempo_event_id → bpm + curve]
- collision_cluster_overrides: dict[cluster_id → strategy + contour_pct]
- Optionally: any Conductor Profile field — overrides profile for this score only
- Optionally: dynamic_tables — per-instrument overrides for this score

3.4 Configuration Cascade

Level	Source
1 — System Defaults	Hardcoded in application. Always present. Sensible starting point.
2 — Conductor Profile	Personal JSON file. Overrides defaults. Travels across scores.
3 — CIL File	Score-specific JSON. Overrides profile. Highest authority.

Resolution rule: for any parameter, the system checks CIL first, then Profile, then System Defaults. The first non-null value wins.

4. Object Definitions

Objects are defined in pipeline order — from top-level container down to embedded sub-objects. All objects implement to_json() and, where appropriate, from_xml() as a classmethod that constructs the object from a music21 element.

4.1 ScoreData

Top-level container. Singleton — one instance per loaded score. Holds all musical data. Serialized to JSON by FastAPI for the React frontend. Contains only musical content — no rendering parameters.

ScoreData	
`title`	string
`composer`	string
`opus`	string | null
`movement`	string | null
`year`	int | null
`engraver_edition`	string | null — edition, source, or preparator
`musicxml_filename`	string
`musicxml_hash`	string — MD5/SHA256 of source file for sync detection
`cil_filename`	string | null
`duration_seconds`	float
`has_unresolved_events`	bool — true if any object needs conductor review
`instruments`	list[InstrumentMeta] — ordered by score_order
`notes`	list[NoteObject] — flat, sorted by time_onset
`sections`	list[ScoreSection] — sorted by time_onset
`dynamic_spans`	list[DynamicSpan] — sorted by time_onset
`tempo_events`	list[TempoEvent] — sorted by time_onset
`collision_clusters`	list[CollisionCluster] — empty at parse time

Functions:

- `to_json() — serialize entire object graph to JSON`
- `from_xml(path) — classmethod, orchestrates full parse pipeline`
- `get_instrument(instrument_id: string) → InstrumentMeta`
- `get_notes_for(instrument_id, bar_start, bar_end) → list[NoteObject]`
- `get_notes_at(time: float) → list[NoteObject]`
- `get_section_at(time: float) → ScoreSection`
- `get_tempo_at(time: float) → float (interpolated bpm)`
- `get_unresolved() → list (all objects with is_auto_analyzed=true)`
- `validate() → ValidationResult`

1 instance per loaded score

4.2 InstrumentMeta

Describes one instrument or part. Stores physical and contextual properties used by the renderer for lane sizing, color assignment, and dynamic normalization. Does not store color — the UI derives color from instrument_family and score_order.

InstrumentMeta	
`instrument_id`	string «PK» — unique identifier, matches NoteObject.instrument_id
`name`	string — e.g. 'Violin I', 'Oboe', 'Horn in F'
`abbreviation`	string — e.g. 'VI.I', 'Ob.', 'Hn.'
`instrument_family`	InstrumentFamilyEnum: STRINGS | WOODWINDS | BRASS | PERCUSSION | KEYBOARD | VOICE
`score_order`	int — 0 = top of score, ascending downward
`transposition`	int (semitones) — 0=C instrument, -2=Bb clarinet, -5=Horn in F
`player_count`	int — number of players on this part (for lane height)
`pitch_range_low`	int (midi) — lowest expected pitch
`pitch_range_high`	int (midi) — highest expected pitch
`dynamic_table`	dict[DynamicMarkingEnum → float] — conductor-adjustable, per instrument

Functions:

- `to_json()`
- `from_xml(part)` — `classmethod, construct from music21 Part object`
- `get_dynamic_value(marking: DynamicMarkingEnum) → float`

1 per instrument/part in score

4.3 NoteObject

The fundamental unit of the foreground layer. One NoteObject per note event per pitch — chords are split into individual NoteObjects. Always stores concert pitch. The renderer reads this object to place and style every visual element in the painting.

NoteObject	
`bar`	int
`beat`	int
`subdivision`	int (0–60) — 0=beat start, 60=next beat; triplet=20, quarter=15

`time_onset`	float (seconds) — primary rendering coordinate
`time_duration`	float (seconds)
`instrument_id`	string «FK → InstrumentMeta»
`voice`	int default=1
`pitch_midi`	int (0–127) — concert pitch always
`pitch_name`	string — e.g. 'G4', 'Bb3' (concert pitch)
`dynamic_marking`	DynamicMarkingEnum | null — PPP|PP|P|MP|MF|F|FF|FFF|FP
`dynamic_value`	float (0.0–1.0) — computed via InstrumentMeta.dynamic_table
`sforzando`	SforzandoEnum | null — SF|FZ|SFZ|FZFZ|SFFZ (relative accent modifier)
`dynamic_span_id`	string | null «FK → DynamicSpan»
`dynamic_span_position`	float (0.0–1.0) | null — pre-computed at parse time
`articulation`	list[ArticulationEnum] — NORMAL|LEGATO|STACCATO| MARCATO|TENUTO|ACCENT|TREMOLO|PORTATO
`technique`	list[TechniqueEnum] — NORMAL|PIZZICATO|COL_LEGNO|SUL_PONTICELLO|HARMONICS| SUL_TASTO|TREMOLO_BOW|SNAP_PIZZICATO|FLAUTANDO
`attack`	AttackEnum — NORMAL | SHARP | SOFT
`release`	ReleaseEnum — NORMAL | CLIPPED | TAPERED
`variant_index`	int | null — assigned by Shape Variant pre-pass
`collision_cluster_id`	string | null «FK → CollisionCluster» — assigned by Collision Resolver

Functions:

- `to_json()`
- `from_xml(note, part)` — classmethod; applies transposition to concert pitch
- `is_colliding_with(other: NoteObject) → bool` — spatial overlap test
- `get_instrument() → InstrumentMeta` — lookup via instrument_id
- `get_dynamic_value() → float` — applies InstrumentMeta.dynamic_table

1 per note event; chords split into individual NoteObjects

4.4 ScoreSection

Encodes the background layer — large-scale formal divisions. Contains a list of HarmonicEvent objects describing the harmonic journey through the section. Simple stable sections have one HarmonicEvent; modulating or complex sections have multiple.

ScoreSection	
`section_id`	string «PK»
`label`	string — free text; UI offers presets: Exposition|Development|Recapitulation|Intro|Coda|Codetta|Bridge|Theme A|Theme B|Other
`is_auto_analyzed`	bool — false once conductor confirms
`start_bar, start_beat`	int
`end_bar, end_beat`	int
`time_onset, time_duration`	float (seconds)
`harmonic_events`	list[HarmonicEvent] — ordered by time_onset; 1 item = stable key

Functions:

- `to_json()`
- `get_color_at(time: float) → HarmonicEvent` — returns active event for UI color mapping
- `get_key_at(time: float) → HarmonicEvent`

HarmonicEvent (embedded in ScoreSection)	
`time_onset, time_duration`	float (seconds)
`key_tonic`	string — e.g. 'G', 'Bb', 'F#'
`key_mode`	enum MAJOR | MINOR
`circle_of_fifths_pos`	int (0–11) — C=0,G=1,D=2,A=3,E=4,B=5,F#=6,Db=7,Ab=8,Eb=9,Bb=10,F=11
`is_transitional`	bool — true during modulation; drives gradient rendering in UI
`is_auto_analyzed`	bool

Functions:

** per ScoreSection — multiple items = modulating section*

4.5 DynamicSpan

Describes a crescendo or diminuendo hairpin. Attached to a specific instrument — the same bar can have multiple DynamicSpans on different instruments simultaneously.

NoteObject.dynamic_span_position is pre-computed at parse time so the renderer never needs to consult this object directly.

DynamicSpan	
`span_id`	string «PK»
`span_type`	enum CRESCENDO | DIMINUENDO
`instrument_id`	string «FK → InstrumentMeta» — one span per instrument per hairpin
`start_bar, start_beat`	int
`end_bar, end_beat`	int
`time_onset, time_duration`	float (seconds)
`dynamic_start`	DynamicMarkingEnum — never null; inferred if not explicit in score
`dynamic_end`	DynamicMarkingEnum — never null; defaults to ±1 level if not explicit
`shape`	ShapeEnum — LINEAR | CONVEX | CONCAVE default=LINEAR
`is_auto_analyzed`	bool — true if dynamic_start or dynamic_end were inferred

Functions:

- `to_json()`
- `from_xml(hairpin, part)` — classmethod; infers boundaries if not explicit
- `get_value_at(position: float) → float` — applies shape curve; position 0.0–1.0

1 per instrument per hairpin; same bar can have multiple spans on different instruments

4.6 TempoEvent

Encodes a tempo marking or change. Used by the parser to convert bar/beat positions to seconds for all other objects. Stored in ScoreData for the Annotation Editor and for the get_tempo_at() function.

TempoEvent	
`tempo_event_id`	string «PK»
`bar, beat`	int
`time_onset`	float (seconds)
`bpm`	float — beats per minute
`beat_unit`	BeatUnitEnum — QUARTER|HALF|EIGHTH|DOTTED_QUARTER|DOTTED_HALF
`transition_type`	enum IMMEDIATE | GRADUAL

`transition_end_bar, transition_end_beat`	int | null
`transition_end_bpm`	float | null — target tempo for gradual changes
`transition_curve`	ShapeEnum | null — LINEAR|CONVEX|CONCAVE
`notation_label`	string | null — text as written: 'Allegro vivace', 'Tempo I'
`is_auto_analyzed`	bool — true if bpm inferred from text label
`is_gradual_unresolved`	bool — true if GRADUAL and transition_end_bpm is null

Functions:

- `to_json()`
- `from_xml(tempo_mark)` — classmethod; resolves 'Tempo I' back-references
- `get_bpm_at(time: float) → float` — interpolates during gradual changes
- `seconds_per_beat() → float` — convenience: 60.0 / bpm

4.7 CollisionCluster

Populated by the Collision Resolver pre-pass after ScoreData is built. Describes a group of notes from different instruments that occupy the same visual space. Each NoteObject in the cluster carries the cluster_id as a foreign key. Conductor can override strategy and blend parameters per cluster in the UI.

CollisionCluster	
`cluster_id`	string «PK»
`note_ids`	list[string] «FK list → NoteObject» — all notes in this cluster
`instrument_ids`	list[string] — for quick family-pair lookups
`time_onset, time_duration`	float (seconds)
`pitch_center`	int (midi) — average pitch of all notes
`pitch_spread`	int (semitones) — 0=pure unison, 1-2=tight cluster, 3+=open voicing
`is_unison`	bool — true if all notes share identical pitch_midi
`family_pairs`	list[string] — all combinations present, normalized alphabetically e.g. 'STRINGS-BRASS'
`strategy`	StrategyEnum — EXPAND|ALTERNATE|BLEND (inherited from UI or overridden)
`contour_pct`	float (0.0–1.0) — blend contour percentage
`shape_override`	bool — true if conductor manually adjusted this cluster
`family_pair_rules`	dict[string → float] | null — per-pair contour overrides
`is_auto_analyzed`	bool

Functions:

- `to_json()`
- `get_dominant_family_pair() → string` — most common family pair in cluster
- `get_strategy() → StrategyEnum` — cluster override if set, else global UI setting
- `get_contour_pct(instrument_a, instrument_b) → float` — checks family_pair_rules, then global table

Populated by Collision Resolver pre-pass; empty list at parse time

4.8 ValidationResult

Returned by ScoreData.validate(). Three-tier system: ERRORS block processing, WARNINGS proceed with flags, SUGGESTIONS draw attention without requiring action.

ValidationResult

errors	list[ValidationMessage] — must fix before proceeding
warnings	list[ValidationMessage] — flag for review; proceed with caution
suggestions	list[ValidationMessage] — draw attention; no action required
is_valid	bool — true if errors is empty
is_clean	bool — true if warnings is empty
is_pristine	bool — true if suggestions is empty too
Functions:	

ValidationMessage (embedded in ValidationResult)	
level	enum ERROR | WARNING | SUGGESTION
code	string — machine-readable e.g. 'MISSING_INSTRUMENT_ID'
message	string — human-readable description
object_type	string — which object type triggered it
object_ref	string | null — id of the specific object
bar	int | null — metric location if applicable
suggestion	string | null — what the conductor might consider (present on SUGGESTION level)
Functions: *Validation runs lightweight on load (hard errors only), full on export request*	

Level	Behavior
ERROR	Stops processing. Must be resolved. Examples: missing instrument_id, negative duration, hash mismatch, circular references.
WARNING	Proceeds with flag. Review before export. Examples: is_auto_analyzed=true, unresolved gradual tempo, orphaned CIL annotation.
SUGGESTION	Proceeds silently. Draws conductor attention. Examples: no dynamic markings in brass, no tempo changes detected, sparse articulation data.

5. Parsing Pipeline

The full pipeline runs when ScoreData.from_xml() is called. It is organized into three phases: Parse, Configure, and Pre-Render.

See: diagram_pipeline.svg for the visual representation.

Step	Action
① Score Parser	Ingests MusicXML via music21. Creates InstrumentMeta and NoteObject for every note. Applies transposition to concert pitch. Computes time_onset via TempoEvent map.
② Harmonic Analyzer	Detects key areas and modulation windows. Creates ScoreSection and HarmonicEvent objects. Marks all as is_auto_analyzed=true.
③ Phrase Segmenter	Identifies formal section boundaries. Assigns preliminary section labels. Marks as is_auto_analyzed=true.
④ Tempo Extractor	Extracts tempo markings and hairpin-style tempo changes. Creates TempoEvent objects. Resolves 'Tempo I' back-references. Flags gradual unresolved events.
⑤ Dynamic Extractor	Extracts hairpin markings. Creates DynamicSpan objects with instrument_id. Infers dynamic_start and dynamic_end where not explicit. Stamps dynamic_span_id and dynamic_span_position on affected NoteObjects.
⑥ Auto-Annotator	Infers attack, release, and technique from articulation markings and score context. Populates NoteObject fields. Marks inferences as is_auto_analyzed=true.
⑦ Hash Check	Computes MD5 hash of MusicXML file. Compares to hash stored in CIL file if present. Generates WARNING if mismatch detected.
⑧ Config Cascade	Resolves all rendering parameters: System Defaults → Conductor Profile → CIL file. Parameters from higher levels override lower ones.
⑨ Shape Variant Pre-pass	Assigns variant_index to every NoteObject using deterministic round-robin seeded by note position. Consults Shape Variant Library.
⑩ Collision Resolver	Detects spatial overlaps in target layout coordinate space. Builds CollisionCluster objects. Stamps collision_cluster_id on affected NoteObjects.
⑪ Validate	Runs full ValidationResult. Populates errors, warnings, and suggestions. Sets has_unresolved_events on ScoreData. UI displays results to conductor.

6. Enum Reference

Enum	Values
DynamicMarkingEnum	PPP · PP · P · MP · MF · F · FF · FFF · FP
SforzandoEnum	SF · FZ · SFZ · FZFZ · SFFZ

ArticulationEnum	NORMAL · LEGATO · STACCATO · MARCATO · TENUTO · ACCENT · TREMOLO · PORTATO
TechniqueEnum	NORMAL · PIZZICATO · COL_LEGNO · SUL_PONTICELLO · HARMONICS · SUL_TASTO · TREMOLO_BOW · SNAP_PIZZICATO · FLAUTANDO
AttackEnum	NORMAL · SHARP · SOFT
ReleaseEnum	NORMAL · CLIPPED · TAPERED
InstrumentFamilyEnum	STRINGS · WOODWINDS · BRASS · PERCUSSION · KEYBOARD · VOICE
ShapeEnum	LINEAR · CONVEX · CONCAVE
StrategyEnum	EXPAND · ALTERNATE · BLEND
BeatUnitEnum	QUARTER · HALF · EIGHTH · DOTTED_QUARTER · DOTTED_HALF
VerticalModeEnum	PITCH_BASED · SCORE_BASED
TemplateEnum	PIANO_ROLL · SPIRAL
ValidationLevelEnum	ERROR · WARNING · SUGGESTION

7. Relationship Summary

Relationship	Cardinality
ScoreData → InstrumentMeta	1 to 1..* (composition)
ScoreData → NoteObject	1 to 0..* (composition)
ScoreData → ScoreSection	1 to 1..* (composition)
ScoreData → DynamicSpan	1 to 0..* (composition)
ScoreData → TempoEvent	1 to 1..* (composition)
ScoreData → CollisionCluster	1 to 0..* (composition; empty at parse time)
NoteObject → InstrumentMeta	Many to 1 (FK: instrument_id)
NoteObject → DynamicSpan	Many to 0..1 (FK: dynamic_span_id)
NoteObject → CollisionCluster	Many to 0..1 (FK: collision_cluster_id)

CollisionCluster → NoteObject	1 to 2..* (FK list: note_ids)
DynamicSpan → InstrumentMeta	Many to 1 (FK: instrument_id)
ScoreSection → HarmonicEvent	1 to 1..* (embedded list)

Document prepared March 2026 · To be reviewed and updated as the data model evolves.